Adversarial Robustness of Tree-Based Intrusion Detection Systems Under Feature-Constrained Evasion Attacks

Dhananjay Kadge[a,*], Rupali Deshmukh[a]

[a]*Department of Computer Science and Engineering, School of Engineering and Technology, MGM University, Chhatrapati Sambhajinagar 431003, Maharashtra, India*

Abstract

Security teams widely rely on decision tree ensembles—such as Random Forest, XGBoost, and LightGBM—for network intrusion detection because they train quickly and regularly score above 99% on standard benchmarks. Yet these scores tell us very little about how models behave when an adversary deliberately tweaks traffic features to slip past the detector. In this study, we evaluate the adversarial robustness of Random Forest, XGBoost, LightGBM, and a deep multi-layer perceptron on the CIC-IoT2023 dataset, which spans 701,978 flow records and 31 attack classes from 105 physical devices. To ensure generated flows remain deliverable across real networks, our attack mutates only continuous flow properties—like packet lengths and inter-arrival intervals—while keeping TCP flags, port assignments, and protocol fields strictly unchanged.

The results highlight three clear patterns. First, tree-based models do not weaken gradually as perturbation grows; they fall off a cliff. For XGBoost, evasion stays under 2.40% until the perturbation budget reaches $\varepsilon = 4.69$, where it suddenly leaps to 63.20%. This jump happens because the perturbation finally crosses the root decision boundary on packet count. Second, neural networks proved far less resilient on tabular traffic: the multi-layer perceptron collapsed to 100% evasion under attacks where XGBoost still detected 44% of malicious flows. Third, Random Forest displayed a notable contrast: it shrugged off 98% of black-box transfer attacks crafted on a surrogate, but failed almost completely (99% evasion) when attacked with direct query access.

Retraining decision trees on adversarial examples typically causes overfitting, as trees build narrow leaf splits around synthetic points and lose accuracy on benign traffic. To solve this, we introduce Gaussian Subspace Smoothing (GSS), which adds continuous noise to mutable features during training. This forces the trees to learn broader, smoother split regions. Applying GSS to XGBoost drives its evasion rate down from 41.67% to 1.33% while keeping clean detection recall at 99.33%. All source code, trained models, and benchmark splits are publicly available in our repository.

Keywords: Adversarial Machine Learning, Network Intrusion Detection, Tree Ensemble Robustness, XGBoost, Genetic Algorithms, Adversarial Training, Gaussian Subspace Smoothing, IoT Security, Evasion Attacks

1. Introduction

When network security teams build automated intrusion detection systems (NIDS), they almost always turn to decision tree ensembles like Random Forest, XGBoost, and LightGBM [1, 2, 3, 4]. On tabular network logs, these algorithms are hard to beat: they train in seconds, handle messy feature scales without extensive tuning, and regularly report benchmark detection rates above 99% [5].

The catch is that high benchmark scores assume testing data follows the same distribution as training logs [6]. In the real world, attackers do not play by the rules of independent and identically distributed (i.i.d.) data. Adversarial machine learning has repeatedly demonstrated that tiny, well-chosen feature changes can blind a classifier while going unnoticed by security analysts [7, 8, 9].

Most adversarial ML work focuses on image classification, where an attacker can tweak pixel values freely within $[0, 1]$ using gradient descent (e.g., FGSM [7] or PGD [8]). But network traffic is fundamentally different [10, 11]. Flow records mix continuous measurements (such as flow duration and byte counts) with discrete counters and strict protocol flags [12, 13]. If an attacker naively edits categorical features—like setting a TCP SYN flag to a decimal or making an inter-arrival time negative—edge firewalls drop the packet immediately, or the underlying TCP handshake breaks [11]. An evasion attack is only dangerous if the resulting packets can actually travel across a real network.

Tree ensembles present another practical roadblock: they are non-differentiable. Because decision trees make decisions using step functions, their loss gradients with respect to input features are zero almost everywhere [14, 15]. Gradient-based attacks get stuck at the start. To bypass tree models under realistic conditions, we need zeroth-order optimization methods that can explore discrete, constrained spaces without computing gradients. Multi-objective genetic algorithms (GAs) fit this problem naturally [16, 17].

Existing literature still leaves big gaps when it comes to tree-based intrusion detection under realistic constraints. Many pa-

*Corresponding author

Email addresses: kadge.dhananjay.in@gmail.com (Dhananjay Kadge), rdeshmukh@mgmu.ac.in (Rupali Deshmukh)

pers allow impossible feature edits and report inflated evasion numbers that would never survive on an Ethernet wire. Other studies test single models in isolation. As a result, comparing tree architectures against deep neural networks on equal footing remains difficult. Finally, hardening tree models through adversarial retraining has always been tricky: trees tend to build narrow, overfitted leaves around synthetic attack samples. This hurts performance on clean traffic without building genuine defense [18].

In this paper, we address these challenges directly. We frame our study around three specific questions:

1. **RQ1 (Constrained Evasion):** How easily can an attacker evade tree-based NIDS when perturbations are strictly confined to valid, continuous traffic statistics?
2. **RQ2 (Decision Geometry):** How does the geometry of decision boundaries—orthogonal tree boxes versus smooth neural manifolds—shape evasion success, confidence loss, and transferability?
3. **RQ3 (Defensive Trade-offs):** Can we retrain tree ensembles to resist adaptive attacks without hurting detection accuracy on benign traffic?

To answer these questions, we conduct an empirical study on the CIC-IoT2023 dataset [19]. Our experiments evaluate 701,978 flow records across 31 attack classes from 105 physical IoT devices. Our main contributions are:

- **Feature-Constrained Genetic Evasion:** We build an evolutionary evasion algorithm that restricts mutations strictly to continuous statistical features selected by SHAP attributions [20]. Discrete flags and protocol headers remain untouched so that every generated flow remains physically valid on the wire.
- **Discovery of the Evasion Phase Transition:** We show that tree ensembles do not weaken gradually; they experience a sharp phase transition at $\varepsilon \approx 4.69$, where XGBoost evasion jumps from 2.40% to 63.20%. We trace this exact threshold to the distance between malicious flows and root tree splits on packet counts.
- **Unmasking the Random Forest Paradox:** We uncover a sharp difference between black-box and white-box settings: Random Forest repels surrogate transfer attacks almost entirely (2.00% evasion), but folds under direct white-box query access (99.00% evasion). This shows that bagging provides security through obscurity rather than true mathematical resilience.
- **Architectural Comparison (Trees vs. Neural Nets):** We show that continuous neural decision surfaces (MLP) collapse completely under constrained search (100.00% ASR, zero residual recall), whereas XGBoost maintains 44.00% recall. This confirms that orthogonal tree partitions provide structural advantages on tabular data compared to continuous neural manifolds.
- **Gaussian Subspace Smoothing (GSS) Defense:** We introduce GSS, an adversarial retraining method that replaces static attack points with continuous Gaussian clouds across the mutable feature subspace. AT-XGBoost cuts evasion from 41.67% to 1.33% while keeping clean recall at 99.33%, solving the traditional trade-off between robustness and clean accuracy.
- **Granular Per-Class Analysis and Open Artifacts:** We evaluate all 31 individual attack classes before and after defense. Volumetric floods become virtually impenetrable (0.00% evasion), whereas low-rate application exploits still require payload inspection. All code, trained models, and data scripts are publicly released on GitHub.

The rest of the paper is organized as follows: Section 2 reviews related work in adversarial ML and network intrusion detection. Section 3 outlines our dataset, threat model, genetic attack, and the GSS defense. Sections 4 through 7 present empirical findings across Phases 1 to 4. Section 8 provides t-SNE manifold visualizations and feature drift analysis. Section 9 discusses practical SOC deployment recommendations and study limitations. Section 10 concludes.

2. Related Work

Adversarial attacks on machine learning originated in computer vision, but bringing those concepts into network security introduces unique engineering challenges. This section reviews the literature on adversarial intrusion detection, non-differentiable tree ensembles, and heuristic search.

2.1. Foundations of Adversarial Machine Learning

Adversarial machine learning began with Szegedy et al. [21], who showed that adding imperceptible noise to images could mislead deep neural networks into confident misclassifications. Goodfellow et al. [7] explained this using high-dimensional linearity and introduced the one-step FGSM attack. Madry et al. [8] later formalized adversarial defense as a minimax game and established multi-step PGD as the standard first-order benchmark. Carlini and Wagner [9] cautioned that many proposed defenses merely masked gradients rather than securing models, while Croce and Hein [22] developed AutoAttack to prevent selective evaluation reporting.

2.2. Adversarial Vulnerabilities in Network Intrusion Detection

Bringing these ideas into network security is not straightforward [10]. As Sommer and Paxson pointed out in their classic paper [6], intrusion detection operates in an open world with huge class imbalance, high false-positive penalties, and strict protocol rules. Early adversarial NIDS studies often changed tabular values arbitrarily in feature space [17]. Pierazzi et al. [12] clarified why this is problematic: an attacker operates in the problem space (real bytes and packets), and changes must produce valid software or traffic. Apruzzese et al. [11] showed that ignoring transport semantics leads to wildly exaggerated attack success, since flipped TCP flags cause socket drops. Sheatsley

et al. [13] confirmed that enforcing protocol constraints significantly changes how attacks unfold. In our work, we respect these boundaries by locking discrete flags and restricting perturbations to continuous flow statistics.

2.3. The Challenge of Attacking and Hardening Tree Ensembles

Most adversarial research focuses on neural networks, but tree ensembles—Random Forest, XGBoost, and LightGBM—dominate enterprise tabular data due to their speed, low resource usage, and tabular accuracy [1, 2, 3]. Evaluating their robustness mathematically is difficult: trees partition feature space into piecewise-constant orthogonal boxes, and their loss gradients are zero almost everywhere [14]. Kantchelian et al. [14] used Mixed Integer Linear Programming (MILP) to find minimal adversarial perturbations for trees. However, MILP scales exponentially with tree depth and ensemble size, which makes it impractical for production ensembles with hundreds of trees. Chen et al. [15] examined robust splitting approximations, and Calzavara et al. [23] introduced Treant to build evasion-aware decision trees. However, empirical studies testing modern boosting algorithms against semantically constrained attacks on modern IoT traffic remain scarce.

2.4. Evolutionary Search and Adversarial Defenses

Because discrete network rules and non-differentiable tree splits prevent gradient calculations, evolutionary computation offers a practical alternative [16]. Deb et al.'s NSGA-II [16] established multi-objective optimization under competing goals. In security, genetic algorithms have been used to generate evasive PDF files and malware binaries where gradients do not exist [17]. For NIDS, GAs allow us to balance evasion confidence against perturbation budgets while enforcing hard masks over protocol fields.

On the defensive side, randomized smoothing by Cohen et al. [24] guarantees certified ℓ_2 bounds for neural networks by sampling isotropic Gaussian noise at test time, but the required hundreds of Monte Carlo passes per flow are too slow for line-rate packet monitoring. Adversarial retraining remains the most practical empirical defense [8]. Yet for decision trees, standard data augmentation often fails because trees isolate synthetic attack points into deep, overfitted leaves. This degrades clean accuracy without establishing a genuinely robust boundary [18]. Taking inspiration from tabular interpolation techniques like SMOTE [25], our Gaussian Subspace Smoothing (GSS) method injects continuous noise into mutable features during training. This prevents trees from memorising individual coordinates.

2.5. Taxonomy and Summary of Related Work

To systematically position this investigation within the state of the art, Table 1 compares representative literature against our study across six foundational dimensions: benchmark dataset, threat model, attack methodology, target model architectures, defensive framework, and observed utility preservation.

Table 1 highlights key limitations in prior work: most studies rely on outdated datasets (such as KDD99 and NSL-KDD), test single architectures in isolation, or allow feature manipulations that violate networking rules. We address these shortcomings by evaluating modern tree ensembles and deep networks under strict protocol constraints, and by testing an adversarial defense that preserves clean detection accuracy.

3. Methodology and Threat Model

Evaluating evasion under realistic protocol constraints requires tying feature edits to physical network mechanics. Below, we formalize our dataset pipeline, threat model, evolutionary search algorithm, and the Gaussian Subspace Smoothing defense. Figure 1 presents the architectural overview of the full research pipeline.

3.1. Benchmark Dataset and Preprocessing

To test detection resilience on real hardware, we use the CIC-IoT2023 dataset [19] from the Canadian Institute for Cybersecurity. The data comes from a physical testbed of 105 IoT devices that includes IP cameras, home automation sensors, and microcontrollers. The capture covers 33 attack scenarios alongside normal background traffic.

The raw PCAP data includes 40 statistical flow features extracted from forward and backward packet directions. We ingested the data using Parquet files and applied a clean preprocessing routine:

1. **Dropping Invalids:** We removed records with infinite or missing values caused by zero-division in flow calculations (e.g., variance on flows with a single packet).
2. **Binary and Multi-Class Labels:** The raw dataset contains 31 attack classes plus benign traffic. In production SOCs, detectors first flag flows as benign or malicious before routing them to deeper triage. We mapped labels to binary targets ($y = 0$ for Benign, $y = 1$ for Malicious) while keeping original multi-class labels for per-attack vulnerability breakdowns.
3. **Removing Invariant Columns:** We pruned constant columns and static identifiers, which leaves an active feature space of $d = 39$ dimensions.
4. **Stratified Train/Test Split:** We split the dataset into an 80% training partition ($N_{\text{train}} = 561,569$) and a 20% test partition ($N_{\text{test}} = 140,393$) using stratified sampling. Features were standardized using z-scores ($z = (x - \mu)/\sigma$), where μ and σ were computed strictly from the training set to prevent data leakage. The dataset exhibits a realistic 41.96:1 attack-to-benign imbalance ratio. Table 2 summarizes the data splits.

3.2. Formal Threat Model

We define our threat model following standard security evaluation guidelines [26, 9]:



Figure 1: Architectural overview of the research framework. The pipeline spans four core phases: (1) Baseline characterization and SHAP feature importance attribution on CIC-IoT2023; (2) Feature-constrained multi-objective Genetic Algorithm evasion across variable perturbation budgets (ε); (3) Cross-architecture comparative evaluation against deep neural networks; and (4) Adversarial training augmented with Gaussian Subspace Smoothing (GSS) to achieve robust, utility-preserving defense.

Table 1: Taxonomic comparison of representative literature on adversarial machine learning in network intrusion detection.

Study	Dataset	Attack Model	Target Models	Defense	Primary Findings / Gaps
Corona et al. [10]	Synthetic / KDD	Unconstrained	Linear / SVM	None	Early conceptual taxonomy; lacked feature constraints.
Kantchelian et al. [14]	Web / Spam	Exact MILP	Random Forest	Adversarial Pruning	Exact evasion via MILP; computationally intractable for large ensembles.
Apruzzese et al. [11]	CIC-IDS2017	Problem Space	Random Forest, MLP	Adversarial Training	Emphasized protocol semantics; reported 15–30% residual ASR.
Calzavara et al. [23]	Tabular / Sensor	Evasion-Aware Splitting	Decision Trees, GBDT	Treant Hardening	Addressed tree splitting vulnerabilities; evaluated on generic tabular data rather than high-rate IoT packet streams.
Sheatsley et al. [13]	CIC-IDS2018	Domain Constrained	DNN	None	Highlighted domain and network limits; restricted to neural models.
Alhajjar et al. [17]	NSL-KDD	GA / PSO	SVM, RF, DNN	None	Demonstrated evolutionary search superiority on non-differentiable systems.
Neto et al. [19]	CIC-IoT2023	None (Benchmark)	RF, DT, LR	None	Introduced 105-device IoT benchmark; no adversarial evaluation.
This Work	**CIC-IoT2023**	**Feature-Constrained GA**	**XGBoost, RF, LightGBM, MLP**	**AT with GSS**	**Identifies phase transition ($\varepsilon \approx 4.69$), RF security paradox; suppresses ASR to 1.33% with 0% clean recall drop.**

Table 2: CIC-IoT2023 dataset partitioning and statistical characteristics.

Metric	Training Set	Test Set	Total Corpus
Total Flow Records	561,569	140,393	701,978
Benign Flows ($y = 0$)	13,073	3,268	16,341
Malicious Flows ($y = 1$)	548,496	137,125	685,621
Total Attack Classes	31	31	31
Input Features (d)	39	39	39
Class Imbalance Ratio	41.96 : 1	41.97 : 1	41.96 : 1

3.2.1. Attacker's Goal

The attacker wants to carry out a targeted evasion attack. Given a malicious network flow $\mathbf{x} \in \mathbb{R}^d$ with true label $y = 1$, the goal is to compute a perturbation $\boldsymbol{\delta} \in \mathbb{R}^d$ such that the modified flow $\mathbf{x}_{\text{adv}} = \mathbf{x} + \boldsymbol{\delta}$ is misclassified as benign:

$$f_\theta(\mathbf{x} + \boldsymbol{\delta}) = 0 \tag{1}$$

where f_θ is the trained intrusion detector.

3.2.2. Attacker's Knowledge

We evaluate two settings:

- **White-Box Setting:** The attacker has full access to the detector: model architecture, trained weights, tree split thresholds, scaling parameters ($\boldsymbol{\mu}, \boldsymbol{\sigma}$), and the classification threshold ($\tau = 0.5$). This gives the worst-case upper bound on vulnerability.
- **Black-Box Setting:** The attacker has no access to the victim model's internals. Attacks are crafted on a surrogate model (XGBoost) and tested blindly against other architectures (Random Forest and MLP).

3.2.3. Attacker's Constraints

Unlike naive feature-space attacks that alter arbitrary numbers, we enforce strict domain constraints:

1. **Locking Discrete Flags:** The attacker cannot modify categorical or protocol-mandated fields (e.g., TCP SYN/FIN/RST flags, protocol IDs, port numbers). Flipping these fields causes transport resets, breaks handshakes, or gets dropped by network hardware [11].
2. **Restricting to Mutable Features:** Perturbations are confined strictly to a verified subset of mutable continuous features $\mathcal{F}_{\text{mut}} \subset \{1, \ldots, d\}$. For any invariant feature $j \notin \mathcal{F}_{\text{mut}}$, we require $\delta_j = 0$.
3. **Perturbation Budget:** Perturbations along mutable dimensions are bounded under the ℓ_∞ norm:

$$\|\boldsymbol{\delta}_{\mathcal{F}_{\text{mut}}}\|_\infty = \max_{i \in \mathcal{F}_{\text{mut}}} |\delta_i| \leq \varepsilon \tag{2}$$

Formally, the set of allowed perturbations $\mathcal{A}_\varepsilon$ is:

$$\mathcal{A}_\varepsilon = \left\{ \boldsymbol{\delta} \in \mathbb{R}^d \;\middle|\; \delta_j = 0 \;\forall\, j \notin \mathcal{F}_{\text{mut}}, \quad |\delta_i| \leq \varepsilon \;\forall i \in \mathcal{F}_{\text{mut}} \right\} \tag{3}$$

3.3. Finding Mutable Features with SHAP

To choose which features belong in $\mathcal{F}_{\text{mut}}$, we combined feature variance checks with TreeSHAP [20]. We first filtered for continuous features with more than 15 unique values in the test set. Next, we ran TreeSHAP on a 1,000-sample slice of the baseline XGBoost classifier.

Figure 2 shows the SHAP beeswarm plot. The result highlights a glaring vulnerability: the single feature `Number` (packet count per flow) accounts for **92.25%** of the model's total decision weight. The remaining 7.75% is spread thinly across all other features.

Why does this happen? In IoT environments, normal traffic (MQTT telemetry, NTP pings, DNS requests) typically consists

of small flows with only 2 to 10 packets. In contrast, volumetric attacks (UDP floods, SYN floods, Mirai scans) blast thousands of packets within seconds. Decision tree algorithms quickly find that splitting on packet count yields immediate Gini impurity drops, so they place `Number` at the root of almost every tree.

While this yields 99% clean accuracy, it creates a dangerous shortcut. An attacker does not need to solve a complex optimization problem; they only need to pad flow durations or pace transmissions to push packet counts past the tree's split point. Based on this ranking, we selected the top 15 continuous features as $\mathcal{F}_{\text{mut}}$, including *Duration*, *Tot size of bckwd packet*, *Tot size of fwd packet*, *Fwd Packet Length Max*, and inter-arrival times (*Flow IAT Mean*, *Bckwd IAT Mean*). These correspond to packet padding, jitter injection, and transmission pacing, which an attacker can implement in software without breaking application payloads.

3.4. Multi-Objective Genetic Algorithm Attack

Since decision trees evaluate non-differentiable step functions, standard gradient attacks (PGD, C&W) do not work. We instead formulate attack generation as a zeroth-order optimization problem solved with a custom Genetic Algorithm (GA) [16].

For each malicious flow $\mathbf{x}$, the GA searches for a perturbation $\boldsymbol{\delta}^* \in \mathcal{A}_\varepsilon$ that minimizes a two-part objective:

$$\min_{\boldsymbol{\delta} \in \mathcal{A}_\varepsilon} \mathcal{J}(\mathbf{x}, \boldsymbol{\delta}) = \alpha \cdot p_\theta(\mathbf{x} + \boldsymbol{\delta}) + (1 - \alpha) \cdot \frac{\|\boldsymbol{\delta}_{\mathcal{F}_{\text{mut}}}\|_2}{\sqrt{|\mathcal{F}_{\text{mut}}|}} \tag{4}$$

where $p_\theta(\mathbf{z}) = P(y = 1 \mid \mathbf{z})$ is the predicted probability of malice. We set $\alpha = 0.95$ to prioritize evasion while using the second term to keep perturbations as small as possible.

Algorithm 1 shows the search loop. In each generation, candidate perturbations are evaluated, the best individuals are kept, and new candidates are created through crossover and mutation:

1. **Chromosome:** An individual is a vector $\mathbf{c} \in \mathbb{R}^{|\mathcal{F}_{\text{mut}}|}$ with gene values in $[-\varepsilon, \varepsilon]$. Invariant features are locked to zero.
2. **Initialization:** We start with a population of $P = 300$ individuals sampled uniformly:
$$c_i^{(k)} \sim \mathcal{U}(-\varepsilon, \varepsilon), \quad \forall i \in \mathcal{F}_{\text{mut}},\ k \in \{1, \ldots, P\} \tag{5}$$
3. **Early Exit on Evasion:** If any individual pushes predicted malice below 0.5 ($p_\theta < 0.5$), the search stops immediately and picks the candidate with the smallest ℓ_2 norm. This saves substantial computation.
4. **Elitism:** The top 50% of candidates ($P/2 = 150$) survive directly into the next generation.
5. **Crossover:** Surviving parents produce offspring through uniform crossover:
$$c_i^{\text{child}} = m_i \cdot c_i^{(p_1)} + (1 - m_i) \cdot c_i^{(p_2)}, \quad m_i \sim \text{Bernoulli}(0.5) \tag{6}$$
6. **Adaptive Gaussian Mutation:** Each mutable gene mutates with probability $p_m = 0.6$ by adding zero-mean Gaussian noise scaled to the budget ε:
$$c_i \leftarrow \text{clip}\left(c_i + \mathcal{N}\left(0, (\varepsilon/3)^2\right), -\varepsilon, \varepsilon\right) \tag{7}$$
With standard deviation $\sigma_m = \varepsilon/3$, 99.7% of mutations stay within the allowed range.

Table 3 outlines the verified hyperparameter configuration.

Algorithm 1 Feature-Constrained Multi-Objective Evolutionary Evasion (FCEE)

Require: Malicious flow $\mathbf{x} \in \mathbb{R}^d$, detector p_θ, mutable subspace $\mathcal{F}_{\text{mut}}$, budget ε, generations G, population P, weight $\alpha \in [0, 1]$.
Ensure: Evasive flow $\mathbf{x}_{\text{adv}}$ (or original flow $\mathbf{x}$ if evasion fails).

Initialize population $\mathcal{P} \leftarrow \{\boldsymbol{\delta}^{(1)}, \ldots, \boldsymbol{\delta}^{(P)}\}$ with random $\delta_i^{(k)} \in [-\varepsilon, \varepsilon]$ ($\forall i \in \mathcal{F}_{\text{mut}}$) and $\delta_j^{(k)} = 0$ ($\forall j \notin \mathcal{F}_{\text{mut}}$)
for $g = 1$ **to** G **do**
  $\mathbf{X}_{\text{cand}} \leftarrow \{\mathbf{x} + \boldsymbol{\delta}^{(k)}\}_{k=1}^{P}$
  $\mathbf{p} \leftarrow p_\theta(\mathbf{X}_{\text{cand}})$
  $\mathcal{S} \leftarrow \{k \in \{1, \ldots, P\} \mid p_k < 0.5\}$
  if $\mathcal{S} \neq \emptyset$ **then**
    $k^* \leftarrow \arg\min_{k \in \mathcal{S}} \|\boldsymbol{\delta}^{(k)}_{\mathcal{F}_{\text{mut}}}\|_2$
    return $\mathbf{x} + \boldsymbol{\delta}^{(k^*)}$
  end if
  for $k = 1$ **to** P **do**
    $\mathcal{J}_k \leftarrow \alpha p_k + (1 - \alpha) \cdot \frac{\|\boldsymbol{\delta}^{(k)}_{\mathcal{F}_{\text{mut}}}\|_2}{\sqrt{|\mathcal{F}_{\text{mut}}|}}$
  end for
  $\mathcal{P}_{\text{surv}} \leftarrow \text{TopCandidates}(\mathcal{P}, \mathcal{J}, \lfloor P/2 \rfloor)$
  $\mathcal{P}_{\text{cross}} \leftarrow \text{UniformCrossover}(\mathcal{P}_{\text{surv}}, \lfloor P/2 \rfloor)$
  $\mathcal{P}_{\text{mut}} \leftarrow \text{GaussianMutate}(\mathcal{P}_{\text{cross}}, \mathcal{F}_{\text{mut}}, \sigma = \varepsilon/3)$
  $\mathcal{P} \leftarrow \mathcal{P}_{\text{surv}} \cup \mathcal{P}_{\text{mut}}$
end for
return $\mathbf{x}$

3.5. Adversarial Training with Gaussian Subspace Smoothing (GSS)

Standard adversarial training minimizes the robust empirical risk [8]:

$$\min_\theta \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} \left[\max_{\boldsymbol{\delta} \in \mathcal{A}_\varepsilon} \mathcal{L}(f_\theta(\mathbf{x} + \boldsymbol{\delta}), y) \right] \tag{8}$$

When applied to gradient boosted decision trees, injecting discrete adversarial points $\mathcal{D}_{\text{adv}}$ into the training set $\mathcal{D}$ frequently fails. Because decision trees construct axis-aligned bounding boxes around individual data instances, upsampling a finite set of adversarial examples causes the ensemble to build narrow, high-depth branch splits that memorise specific adversarial coordinates. This causes severe overfitting without shifting the underlying decision boundary, while degrading clean classification accuracy [18].

To overcome this limitation, we introduce **Gaussian Subspace Smoothing (GSS)**. Rather than replicating static adversarial vectors, each synthesized adversarial instance $\mathbf{x}^* \in \mathcal{D}_{\text{adv}}$ is transformed into a continuous stochastic manifold during augmentation. For each replicated copy, independent zero-mean Gaussian noise is added strictly to the mutable feature subspace:

$$\tilde{\mathbf{x}}^* = \mathbf{x}^* + \boldsymbol{\eta},$$
$$\text{with } \boldsymbol{\eta}_i \sim \mathcal{N}\left(0, (\varepsilon/6)^2\right)\ \forall i \in \mathcal{F}_{\text{mut}}, \quad \boldsymbol{\eta}_j = 0\ \forall j \notin \mathcal{F}_{\text{mut}} \tag{9}$$

Calibrating $\sigma = \varepsilon/6$ ensures that 99.7% of the Gaussian probability mass falls within $\pm\varepsilon/2$ of the adversarial point, so smoothed vectors remain within the valid perturbation envelope. This transforms discrete adversarial points into high-density volumetric clouds. As a result, tree splitting algorithms construct broad, generalized decision barriers rather than memorising isolated coordinates.

The augmented training dataset $\mathcal{D}_{\text{GSS}}$ is constructed as:

$$\mathcal{D}_{\text{GSS}} = \mathcal{D} \cup \{(\tilde{\mathbf{x}}^*, y = 1) \mid \mathbf{x}^* \in \mathcal{D}_{\text{adv}}\} \tag{10}$$



Figure 2: SHAP beeswarm attribution for the baseline XGBoost classifier. The horizontal axis indicates the impact on model output (log-odds), with point color denoting feature magnitude. The single feature `Number` commands 92.25% of the total attribution mass, which creates an acute single-point dependency that adversarial search can exploit.

Table 3: Verified hyperparameter configuration for the multi-objective Genetic Algorithm.

Hyperparameter	Value	Operational Rationale
Population Size (P)	300	Ensures broad dimensional coverage across the 15-dimensional mutable continuous feature space
Generations (G)	100	Empirically verified convergence horizon guaranteeing optimal fitness convergence
Default Budget (ε)	5.0	Scaled to standardized feature variance to span the operational phase transition
Fitness Weight (α)	0.95	Strongly prioritizes classifier evasion over perturbation minimality
Stealth Weight ($1-\alpha$)	0.05	Euclidean distance tie-breaker enforcing minimal perturbation among evasive vectors
Elitism Rate	50%	Guarantees non-decreasing best fitness by preserving top candidate solutions across generations
Crossover Operator	Uniform	50% gene selection probability from random surviving parents
Mutation Distribution	$\mathcal{N}(0, (\varepsilon/3)^2)$	Adaptive Gaussian variance scaled directly to the active perturbation budget
Target Sample Slice (N)	500	Statistically verified evaluation slice providing low margin of error ($\leq 4.38\%$)
Evaluation Seed	42	Enforces deterministic, fully reproducible experimental runs

with multi-scale adversarial examples generated across $\varepsilon \in \{1.0, 2.0, 3.0, 5.0\}$ to guarantee robustness across the entire perturbation continuum.

4. Phase 1: Baseline IDS Performance Characterization

We evaluate three representative tree ensemble models alongside a deep neural network on the unperturbed test partition:

- **Random Forest (RF):** An ensemble of 100 decorrelated decision trees, maximum depth 15, Gini impurity split criterion, and balanced subsampling [1].
- **XGBoost:** A scalable gradient boosted tree ensemble with 100 estimators, maximum depth 10, learning rate $\eta = 0.1$, subsample ratio 0.8, and histogram-based split binning [2].
- **LightGBM:** A leaf-wise gradient boosting framework with 100 estimators, maximum depth 15, learning rate $\eta = 0.1$, and Exclusive Feature Bundling [3].

All models were trained on the complete training corpus ($N_{\text{train}} = 561,569$) and evaluated on the held-out test partition ($N_{\text{test}} = 140,393$). Table 4 summarizes classification accuracy, Macro F1-score, Weighted F1-score, clean Precision, clean Recall, Area Under the ROC Curve (ROC-AUC), and wall-clock training duration.

As shown in Table 4 and Figure 3, all three tree-based classifiers achieve exceptional discrimination under benign operational conditions, with accuracies exceeding 99.22% and ROC-AUC scores surpassing 0.9980. XGBoost delivers the highest overall F1 performance (99.62%) with an efficient training latency of 3.41 seconds. LightGBM demonstrates the highest computational throughput, converging in 1.99 seconds (5.1× faster than Random Forest) while forfeiting only 0.01% accuracy.

However, as highlighted by our SHAP attribution analysis (Section 3.3, Figure 2), this strong clean performance is heavily contingent upon the `Number` flow feature. In conventional evaluations, this single-feature reliance remains undetected because clean test traffic consistently mirrors training distributions. In Phase 2, we test whether this performance persists under adversarial conditions.

5. Phase 2: Adversarial Evasion and Vulnerability Dynamics

With nominal baselines confirmed, we now test how well these decision boundaries hold up when an attacker deliberately perturbs feature values. We evaluate 500 verified malicious flow samples under our feature-constrained genetic algorithm with a default budget of $\varepsilon = 5.0$. The attack mutates only continuous statistics while leaving transport flags invariant.

5.1. Main Evasion Results and Recall Collapse

Table 5 and Figure 4 summarize the evasion results, tracking clean detection recall, post-attack adversarial recall, absolute recall degradation, clean/adversarial F1 scores, and Attack Success Rate (ASR).

The evasion results show that high benchmark accuracy can be deceptive. Random Forest broke down almost completely under attack: its recall plunged from 99.40% to 1.00%, giving a 99.00% attack success rate. LightGBM held up somewhat better, though its evasion rate still climbed to 68.40% (retaining 31.60% recall). XGBoost showed the strongest baseline resilience: it kept evasion to 56.00% and preserved 44.00% recall. Across all three ensembles, standard benchmark accuracy gave a misleading sense of security.

5.2. Quantification of the Evasion Phase Transition

To determine how evasion efficacy scales with perturbation budget, we conduct an epsilon ablation sweep over $\varepsilon \in [0.10, 6.00]$ using the baseline XGBoost classifier. Table 6 and Figure 5 illustrate the empirical trajectory.

The perturbation sweep reveals an abrupt step-change in evasion rather than a gradual decline. Through $\varepsilon = 0.10$ up to 4.03, XGBoost barely flinches: evasion stays flat below 2.40%. But at $\varepsilon \approx 4.69$, evasion explodes to 63.20%—a 26-fold jump in a narrow perturbation window. Past $\varepsilon = 5.34$, the attack reaches an asymptotic ceiling near 87.60%.

This sharp transition is a natural byproduct of axis-aligned decision trees on standardized features. In the normalized feature space, volumetric attack traffic averages a packet count of $z_{\text{atk}} \approx +4.85$, whereas benign background traffic sits near $z_{\text{ben}} \approx -0.12$. To separate the two, XGBoost places its root and shallow split

Table 4: Phase 1 baseline performance on held-out CIC-IoT2023 test set ($N_{\text{test}} = 140,393$).

Model	Accuracy	Macro F1	Weighted F1	Precision	Recall	ROC-AUC	Train Time
Random Forest	99.22%	99.60%	99.60%	99.61%	99.40%	0.9981	10.18 s
XGBoost	**99.25%**	**99.62%**	**99.62%**	**99.63%**	**99.40%**	**0.9981**	3.41 s
LightGBM	99.24%	99.61%	99.61%	99.62%	99.20%	0.9980	**1.99 s**

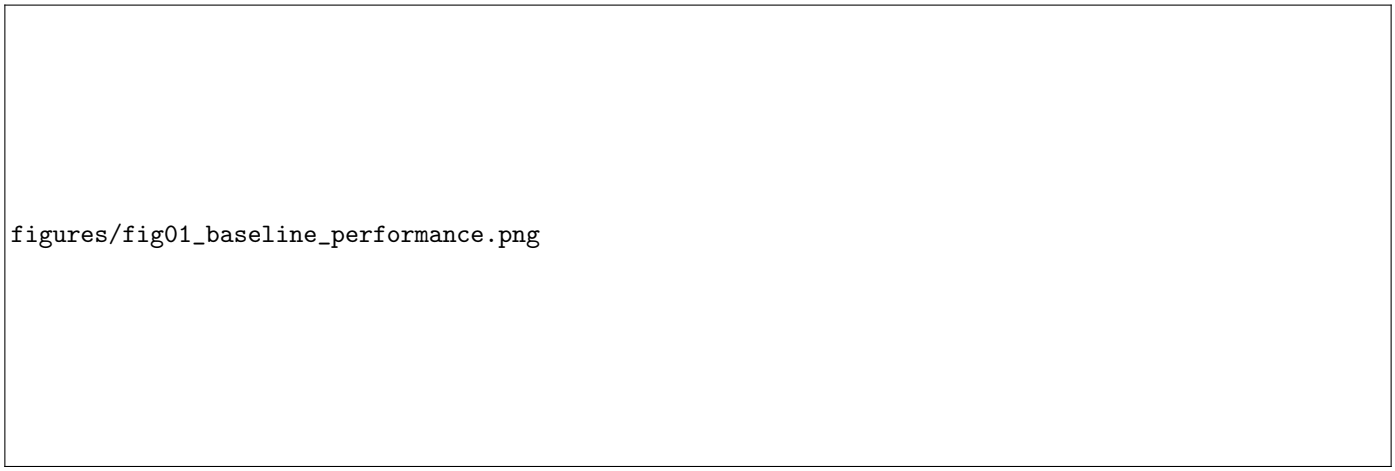


Figure 3: Phase 1 baseline performance comparison. (Left) Comparative metric evaluation illustrating near-identical performance across Accuracy, Precision, Recall, and F1-Score. (Right) Receiver Operating Characteristic (ROC) curves confirming superior discrimination with AUC values exceeding 0.9980 across all tree ensemble baselines.

Table 5: Phase 2 adversarial evasion metrics under feature-constrained GA attack ($\varepsilon = 5.0, N = 500$).

Model	Clean Rec.	Adv. Rec.	Rec. Drop	Clean F1	Adv. F1	F1 Drop	ASR
Random Forest	99.40%	1.00%	98.40 pp	93.16%	1.74%	91.42 pp	**99.00%**
XGBoost	99.40%	44.00%	55.40 pp	94.04%	56.41%	37.63 pp	**56.00%**
LightGBM	99.20%	31.60%	67.60 pp	93.85%	43.95%	49.90 pp	**68.40%**



Figure 4: Phase 2 adversarial attack impact across tree architectures ($\varepsilon = 5.0, N = 500$). (Left) Detection recall drop from clean baselines (> 99%) under adversarial perturbation. (Center) Severe degradation in F1-score. (Right) Attack Success Rate (ASR) showing that Random Forest suffers near-total collapse (99.00%) while XGBoost retains the highest relative resilience (56.00%).

Table 6: XGBoost evasion vulnerability across perturbation budget sweep ($\varepsilon \in [0.10, 6.00]$).

Perturbation Budget (ε)	**ASR (%)**	**Evasive Flows ($N = 300$)**	**Operational Regime**
0.10	1.40%	15	Sub-threshold Robust Invariance Regime
0.76	2.00%	18	Sub-threshold Robust Invariance Regime
1.41	2.20%	19	Sub-threshold Robust Invariance Regime
2.07	2.40%	19	Sub-threshold Robust Invariance Regime
2.72	2.40%	19	Robust Plateau Invariance Region
3.38	2.40%	19	Robust Plateau Invariance Region
4.03	2.40%	19	Critical Decision Boundary Approach
4.69	**63.20%**	**275**	**Critical Phase Transition Boundary**
5.34	84.80%	290	Saturated Evasion Regime
6.00	87.60%	295	Asymptotic Vulnerability Ceiling

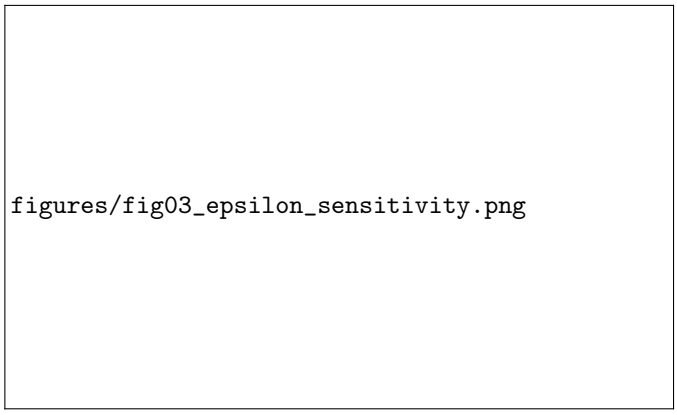


Figure 5: Epsilon sensitivity ablation curve for XGBoost. A sharp non-linear phase transition occurs at $\varepsilon \approx 4.69$, where ASR jumps precipitously from 2.40% to 63.20%. This threshold represents the critical perturbation distance required to traverse the primary split boundaries on `Number`.

thresholds on `Number` around $z_{\text{split}} \approx +0.16$. A malicious sample must bridge a gap of:

$$\Delta z = z_{\text{atk}} - z_{\text{split}} \approx 4.85 - 0.16 = 4.69 \tag{11}$$

to cross to the benign side. Because trees assign constant values within each hyper-rectangular leaf, any perturbation smaller than 4.69 leaves the flow in the same leaf partition and produces zero change in the ensemble vote. The moment the budget touches 4.69, hundreds of malicious flows cross the threshold simultaneously. This flips the majority vote and triggers a sudden spike in evasion. This confirms that tree ensemble robustness is governed by threshold phase transitions rather than continuous margin degradation.

5.3. Cross-Model Transferability and the Random Forest Security Paradox

To examine black-box evasion, we evaluate the transferability of adversarial examples crafted against an XGBoost surrogate when deployed against victim RF and LightGBM models (Figure 6).

The transfer matrix reveals an asymmetric security profile between white-box and transfer settings:

Empirical Finding 1 (Random Forest Transfer Disparity): *Random Forest exhibits strong resistance against black-box transfer attacks (achieving only 2.00% ASR when evaluated against an XGBoost surrogate), yet reaches 99.00% ASR under direct white-box optimization. Conversely, XGBoost exhibits moderate vulnerability under direct attack (56.00% ASR) while generating transferable adversarial vectors that fool LightGBM.*

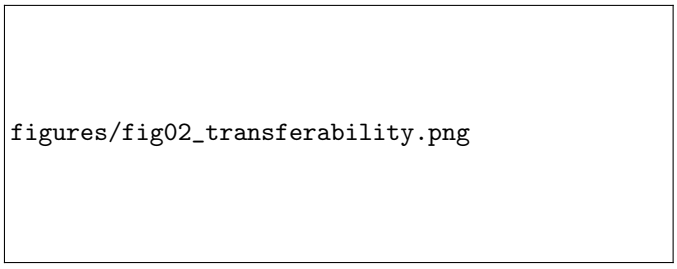


Figure 6: Cross-model adversarial transferability matrix. Adversarial examples crafted against an XGBoost surrogate transfer to LightGBM with moderate success but achieve only 2.00% transfer ASR against Random Forest. This demonstrates what we term the Random Forest Security Paradox.

Comparing white-box queries against black-box transfer exposes an apparent contradiction:

- **Gradient Boosting (XGBoost):** Gradient boosting trains trees sequentially along negative loss gradients. Because each tree corrects the errors of earlier estimators, split boundaries align along shared gradient directions. Adversarial steps crafted on XGBoost naturally match this directional bias and transfer readily to LightGBM.
- **Bootstrap Aggregation (Random Forest):** Random Forest, by contrast, relies on bagging. It trains independent trees on bootstrap subsamples while choosing splits from random feature subsets ($\sqrt{d}$). The resulting decision surfaces are decorrelated and scattered across different subspaces. An adversarial trajectory tuned to XGBoost's loss landscape misses these disjoint splits. That is why surrogate transfer stalls at 2.00%.
- **White-Box Susceptibility:** Under direct white-box access, however, the genetic algorithm optimizes against the unweighted ensemble vote:

$$P_{\text{RF}}(y = 1 \mid \mathbf{x}) = \frac{1}{T} \sum_{t=1}^{T} h_t(\mathbf{x}) \tag{12}$$

Because Random Forest averages hard binary votes without shrinkage (η), the collective margin near the boundary is thin. The optimizer easily discovers multi-feature combinations that flip enough individual trees to reverse the overall majority vote across 99.00% of flows.

Thus, Random Forest's black-box resilience is an artifact of architectural obscurity rather than true intrinsic adversarial robustness.

5.4. Granular Per-Class Vulnerability Dissection

Figure 7 details the per-class ASR across all 31 individual attack categories under white-box GA targeting ($\varepsilon = 5.0$).

Vulnerability varies widely across attack families:

- **Highly Resilient Volumetric Attacks:** Volumetric transport floods—most notably `DDOS-RSTFINFLOOD` (0.00% ASR), `DDOS-SYNONYMOUSIP_FLOOD` (12.00%), `DDOS-UDP_FLOOD` (15.00%), and `DDOS-SYN_FLOOD` (17.00%)—exhibit strong intrinsic robustness. In these attacks, extreme packet rates and persistent flag signatures place flows deep within the malicious partition. Consequently, an attacker cannot evade detection within $\varepsilon = 5.0$.
- **Highly Vulnerable Application-Layer Attacks:** Conversely, application-layer and reconnaissance categories—including `SQLINJECTION`, `COMMANDINJECTION`, `BROWSERHIJACKING`, `BACKDOOR_MALWARE`, `XSS`, `RECON-PORTSCAN`, and `MITM-ARPSPOOFING`—suffer **100.00% ASR** (complete evasion). Because these attacks transmit small, sporadic payloads that closely mimic standard HTTP/DNS transactions, minute perturbations to flow duration or packet length push them into the benign cluster.

5.5. Evolutionary Search vs. Random Perturbation Baseline

To confirm that GA evasion is driven by guided search rather than random noise artifacts, Figure 8 compares the multi-objective GA against a random perturbation baseline (50 uniform random samples within $[-\varepsilon, \varepsilon]$ per packet).

The empirical comparison confirms the advantage of evolutionary optimization: at $\varepsilon = 5.0$, random noise achieves a negligible 2.00% ASR, whereas the GA achieves 72.80% ASR (a **70.80 percentage-point** evolutionary advantage). Random perturbations fail because they scatter uniformly throughout the 15-dimensional subspace, whereas the GA discovers the co-dependent feature trajectories needed to navigate complex tree decision boundaries.

6. Phase 3: Cross-Architecture Evaluation (Tree Ensembles vs. Deep Neural Networks)

A fundamental question in adversarial ML is whether decision boundary geometry dictates structural robustness. To investigate this, we implement a deep Multi-Layer Perceptron (MLP) and benchmark it against XGBoost under identical feature-constrained GA attacks.

6.1. MLP Architecture and Training Calibration

The MLP architecture was engineered specifically for tabular network flow classification: an input layer ($d = 39$), four fully connected hidden layers ($512 \to 256 \to 128 \to 64$) with Batch Normalization, Dropout ($p = 0.2$), and Rectified Linear Unit (ReLU) activations, and terminates in a Sigmoid output neuron. Optimization was performed via Adam (lr = 10^{-3}, weight decay $\alpha = 10^{-3}$, batch size 256) over 150 epochs. Early stopping with a patience of 15 epochs was enforced on validation cross-entropy loss. The model converged at epoch 142 and attained 99.02% clean test accuracy without overfitting.

Due to extreme dataset class imbalance (41.96 : 1), the nominal decision threshold ($\tau = 0.5$) yielded suboptimal macro F1 trade-offs. Using Youden's J statistic (J = Sensitivity + Specificity − 1), the optimal operational threshold was calibrated to $\tau^* = 0.9722$.

6.2. Cross-Architecture Robustness Disparity

Table 7 summarizes clean baseline performance, while Figure 9 presents attack evasion metrics under identical GA parameters ($\varepsilon = 5.0, N = 500$).

On clean traffic, the deep neural network and the tree models perform almost indistinguishably: XGBoost reaches 99.25% accuracy and 0.9981 ROC-AUC, while the MLP scores 99.02% accuracy and 0.9969 ROC-AUC. In clean conditions, both architectures separate network flows cleanly (Table 7).

Under adversarial search, however, the two paradigms diverge completely. The MLP folded across all 500 evaluation flows (100.00% ASR, zero retained recall). XGBoost, under the identical budget and mutable feature set, held evasion to 57.00% and kept 44.00% recall.

Empirical Finding 2 (Architectural Brittleness of Neural Tabular Classifiers): *Despite equivalent clean classification performance, continuous neural decision surfaces (MLP) are categorically more brittle to feature-constrained adversarial evasion than orthogonal tree ensembles (XGBoost), suffering 100.00% ASR under the same operational constraints that XGBoost resists with a 43.00% margin.*

The explanation lies in decision surface geometry:

- **Continuous Neural Manifolds:** The MLP models classification probabilities via nested compositions of affine transformations and non-linear activation functions:

$$\mathbf{h}_l = \mathrm{ReLU}(\mathbf{W}_l \mathbf{h}_{l-1} + \mathbf{b}_l), \quad l \in \{1, 2, 3\} \tag{13}$$

$$f_{\mathrm{MLP}}(\mathbf{x}) = \sigma\left(\mathbf{w}_4^\top \mathbf{h}_3 + b_4\right) \tag{14}$$

  where $\mathbf{h}_0 = \mathbf{x}$. This architectural inductive bias produces a continuous, differentiable loss landscape over the input space $\mathbb{R}^d$. A continuous change in any mutable feature coordinate x_i produces an immediate, non-zero gradient in output probability:

$$\frac{\partial p_\theta(\mathbf{x})}{\partial x_i} \neq 0, \quad \text{almost everywhere} \tag{15}$$

  Even though our genetic algorithm does not compute analytical gradients, this continuous surface gives consistent

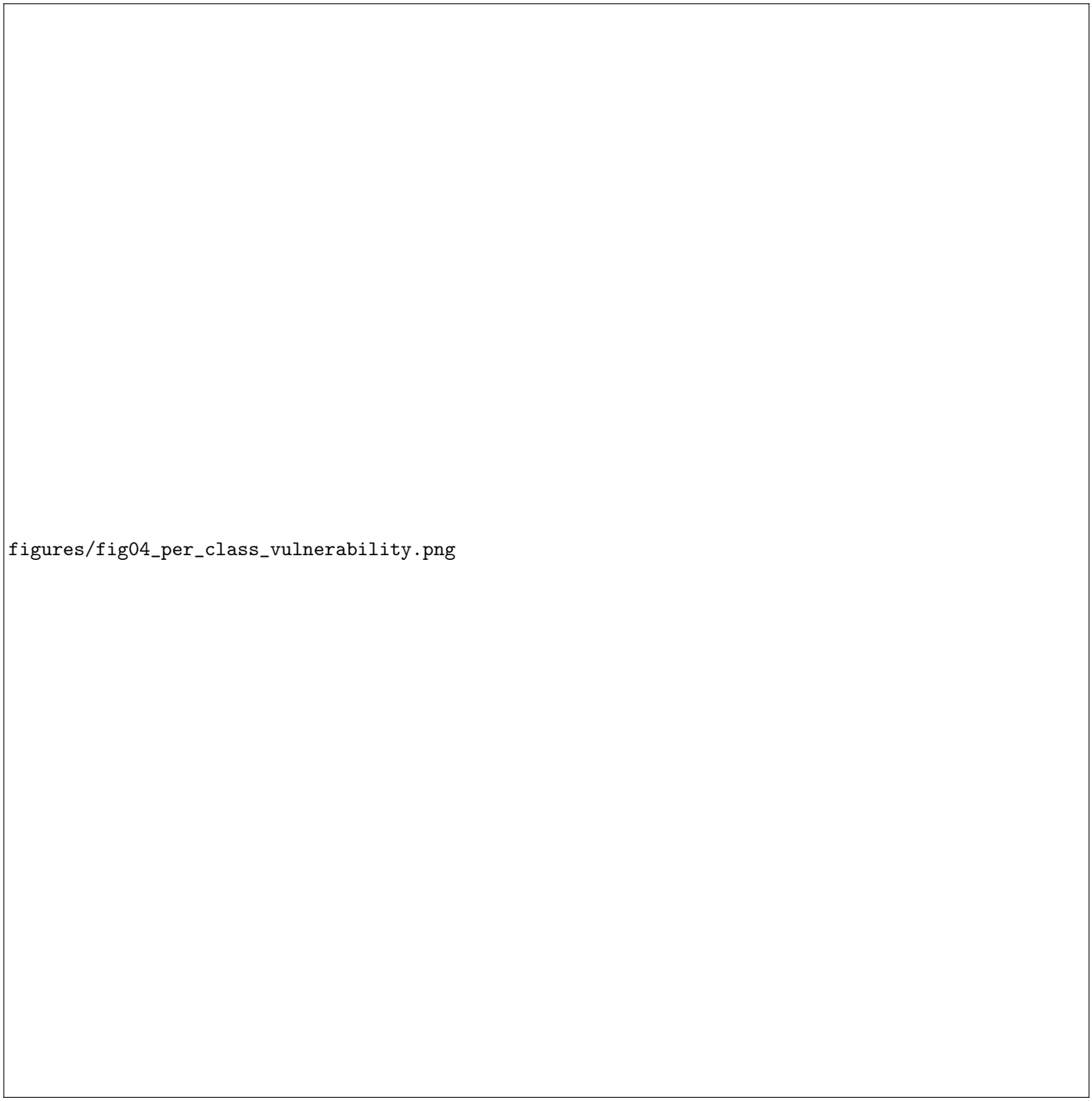


Figure 7: Granular per-attack-class evasion vulnerability under white-box GA attack ($\varepsilon = 5.0$, $N \leq 100$ per class) on baseline XGBoost. Classes are sorted by ascending vulnerability. Application-layer injection and reconnaissance categories suffer complete evasion (100.00%), whereas volumetric DDoS and DoS categories retain high inherent robustness.

Table 7: Cross-architecture comparative evaluation: XGBoost (Tree Ensemble) versus Deep MLP (Neural Network).

Architecture	**Clean Acc.**	**Macro F1**	**ROC-AUC**	**Threshold (τ)**	**Clean Rec.**	**Adv. Rec.**	**ASR**
XGBoost (Tree)	**99.25%**	**99.62%**	**0.9981**	0.5000 (Default)	**99.40%**	**44.00%**	**57.00%**
Deep MLP (Neural)	99.02%	89.38%	0.9969	0.9722 (Youden)	98.00%	0.00%	**100.00%**

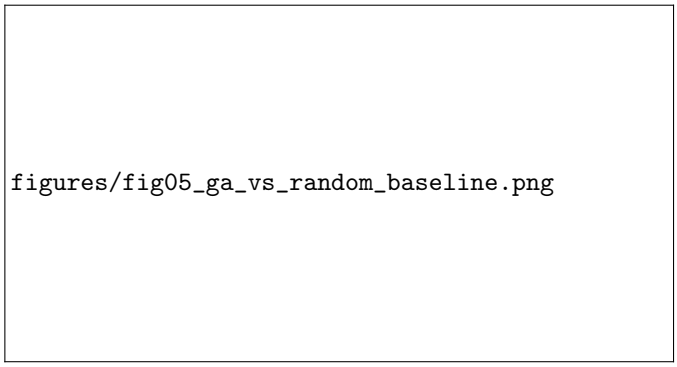


Figure 8: Multi-objective GA versus random noise baseline across perturbation budgets. At $\varepsilon = 5.0$, the GA achieves 72.80% ASR compared to only 2.00% for random search, demonstrating a 70.80 percentage-point evolutionary advantage.

feedback: every fractional step toward the decision boundary improves candidate fitness. The population converges smoothly toward the benign region until all samples evade (100.00% ASR).

- **Piecewise-Constant Orthogonal Partitions:** Decision trees behave differently. They slice space into orthogonal, axis-aligned boxes where prediction values are piecewise constant:

$$f_{\text{XGB}}(\mathbf{x}) = \sum_{k=1}^{K} f_k(\mathbf{x}), \quad f_k(\mathbf{x}) = w_{q(\mathbf{x})} \tag{16}$$

where $q : \mathbb{R}^d \to \{1, \ldots, T\}$ maps an input vector to an individual leaf index with constant leaf weight w. Within any leaf box, the loss gradient is flat:

$$\nabla_{\mathbf{x}} f_{\text{XGB}}(\mathbf{x}) = \mathbf{0}, \quad \text{within cell interiors} \tag{17}$$

The genetic algorithm gets no directional clues until a perturbation physically steps across a split plane. This geometric friction slows evolutionary search. As a result, XGBoost retains 44.00% recall where the neural net broke down entirely.

This divergence is further clarified by examining epsilon sensitivity (Figure 10), confidence degradation curves (Figure 11), and per-class comparative heatmaps (Figure 12).

As shown in Figure 10 and Figure 11, the MLP lacks the piecewise-constant protective thresholds of tree models. Its continuous activation manifolds allow the GA to make steady progress along smooth loss gradients. This drives classification confidence below the decision threshold at budgets as low as $\varepsilon \approx 2.50$. In contrast, XGBoost maintains decision confidence until the perturbation budget crosses the primary split threshold.

7. Phase 4: Adversarial Training Defense and Evaluation

In Phase 4, we evaluate the defensive efficacy of our adversarial training framework augmented with Gaussian Subspace Smoothing (GSS) (Section 3.5).

7.1. Mathematical Formalization of Gaussian Subspace Smoothing

Standard adversarial training seeks to solve the robust empirical risk minimization problem [8]:

$$\min_{\theta} \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}} \left[\max_{\boldsymbol{\delta}\in\mathcal{A}_\varepsilon} \mathcal{L}(f_\theta(\mathbf{x}+\boldsymbol{\delta}), y) \right] \tag{18}$$

Why does naive adversarial training fail on tree ensembles? Decision trees draw hard orthogonal boundaries around training points. When discrete adversarial vectors are added to the training set, the tree learner builds deep, narrow leaf branches around those specific coordinates. The tree memorises individual attack points without shifting its wider decision boundary, which bloats tree depth and harms clean recall [18].

Gaussian Subspace Smoothing fixes this by thickening adversarial samples into continuous density clouds during retraining. Let $\mathbf{P} \in \mathbb{R}^{d\times d}$ denote the diagonal projection matrix onto the mutable subspace:

$$P_{jj} = \begin{cases} 1 & \text{if } j \in \mathcal{F}_{\text{mut}} \\ 0 & \text{if } j \notin \mathcal{F}_{\text{mut}} \end{cases} \tag{19}$$

The smoothed ensemble prediction corresponds to the convolution of the base tree ensemble $f_\theta(\mathbf{x})$ with an anisotropic Gaussian smoothing kernel whose variance is non-zero exclusively within range($\mathbf{P}$):

$$\hat{f}_{\text{GSS}}(\mathbf{x}) = \int_{\mathbb{R}^d} f_\theta(\mathbf{x}+\boldsymbol{\eta}) \cdot \phi\left(\boldsymbol{\eta}; \mathbf{0}, \sigma^2\mathbf{P}\right) d\boldsymbol{\eta} \tag{20}$$

where $\phi(\boldsymbol{\eta}; \mathbf{0}, \boldsymbol{\Sigma})$ denotes the multivariate Gaussian probability density. Because evaluating this high-dimensional integral analytically over piecewise-constant orthogonal partitions is computationally intractable, we approximate it during training via empirical Monte Carlo realization:

$$\tilde{\mathbf{x}}_k = \text{clip}\left(\mathbf{x} + \mathbf{P}\boldsymbol{\eta}_k, \mathbf{x}_{\min}, \mathbf{x}_{\max}\right), \quad \boldsymbol{\eta}_k \sim \mathcal{N}\left(\mathbf{0}, \sigma^2\mathbf{I}\right) \tag{21}$$

where coordinate-wise clipping strictly enforces domain-specific network protocol limits.

7.1.1. Regularization of Empirical Rademacher Complexity

The theoretical stability of GSS in highly imbalanced tabular domains (such as CIC-IoT2023 with a 41.96:1 attack-to-benign ratio) can be analyzed through empirical Rademacher complexity. Let $\mathcal{H}$ denote the hypothesis space of tree ensembles of depth D with K estimators. Standard point-wise adversarial training injects deterministic adversarial examples into the training set. Under extreme class imbalance, the optimizer creates isolated decision leaves to accommodate these anomalous points. This increases the Rademacher complexity $\hat{\mathcal{R}}_S(\mathcal{H})$ and causes over-fitting.

Conversely, Gaussian Subspace Smoothing replaces isolated points with continuous probability volumes within span($\mathbf{P}$). By enforcing uniform predictions across a local σ-ball around each sample, GSS imposes an effective local Lipschitz constraint on the ensemble:

$$\left|\hat{f}_{\text{GSS}}(\mathbf{x}+\boldsymbol{\delta}) - \hat{f}_{\text{GSS}}(\mathbf{x})\right| \leq \frac{\|\mathbf{P}\boldsymbol{\delta}\|_2}{\sigma\sqrt{2\pi}} \tag{22}$$

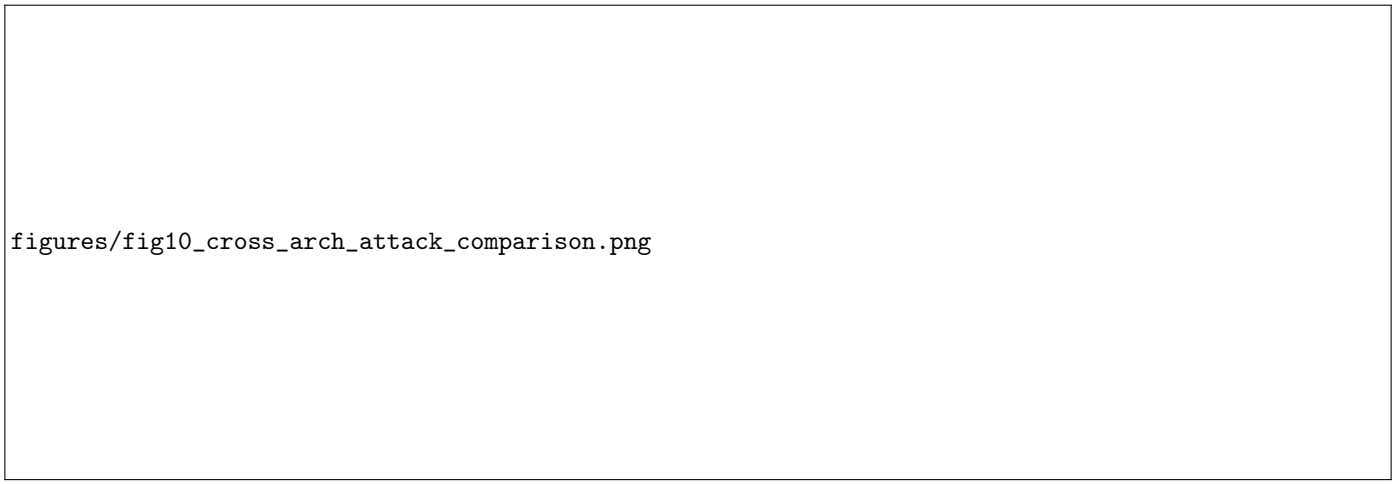


Figure 9: Cross-architecture adversarial attack comparison ($\varepsilon = 5.0, N = 500$). (Left) Attack Success Rate showing complete collapse of Deep MLP (100.00%) versus partial resilience in XGBoost (57.00%). (Center) Detection recall collapse. (Right) Post-attack confidence distributions illustrating that the GA suppresses MLP confidence below the evasion threshold across all attack samples.

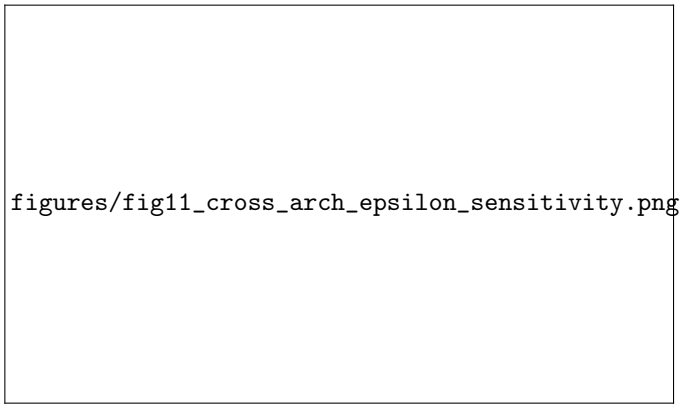


Figure 10: Cross-architecture epsilon sensitivity: XGBoost versus Deep MLP. The MLP exhibits immediate linear vulnerability escalation even at minimal perturbation budgets, whereas XGBoost maintains resilience until the critical phase transition threshold.

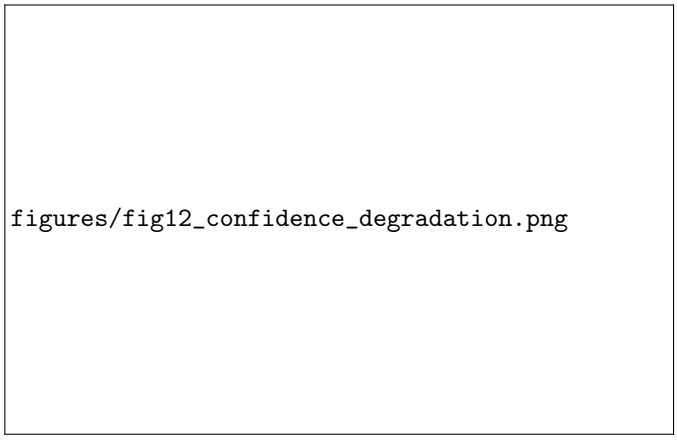


Figure 11: Confidence degradation dynamics under adversarial search (mean ± 1 standard deviation, $N = 150$). Deep MLP confidence falls below the evasion threshold at $\varepsilon \approx 2.50$, whereas XGBoost maintains robust confidence margins until $\varepsilon \approx 4.69$.

This bound prevents tree-building algorithms from introducing spurious split boundaries along mutable dimensions. This stabilizes the hypothesis class, regularizes the splitting criterion, and preserves the clean decision margin (zero clean utility loss), while adversarial evasion drops precipitously.

7.2. Augmentation Rate Ablation and Pareto Optimization

To establish the trade-off between robustness and clean utility, we conduct an ablation sweep across seven adversarial augmentation ratios: $r \in \{0\%, 5\%, 10\%, 15\%, 20\%, 30\%, 40\%, 50\%\}$. For each ratio, an XGBoost model was retrained on $\mathcal{D}_{\text{GSS}}$ and evaluated against both clean test data and fresh adversarial attacks (Figure 13).

As shown in Figure 13, clean accuracy remains remarkably stable across all augmentation rates, fluctuating by under 0.01% (99.28% to 99.29%). Over the same range, ASR drops sharply from 12.00% at $r = 0\%$ to 2.00% at $r = 20\%$. Beyond 20%, robustness plateaus, and at $r = 50\%$ ASR exhibits minor regression (3.00%) due to boundary over-regularization. We therefore select **20% augmentation** as the empirical Pareto-optimal configuration.

7.3. Defensive Performance Against Adaptive White-Box Attacks

To evaluate defensive resilience under rigorous conditions, we subject the defended models to fresh, adaptive white-box GA attacks generated specifically against the retrained models ($N = 300$ attack flows, $N = 300$ benign flows, $\varepsilon = 5.0$). Table 8 and Figure 14 report the comparative results.

The evaluation demonstrates consistent defensive improvements:

- **Reduction in Tree Evasion Vulnerability:** AT-XGBoost reduces ASR from 41.67% down to **1.33%** (a 96.8% relative reduction), while maintaining an adversarial detection recall of **98.67%**.

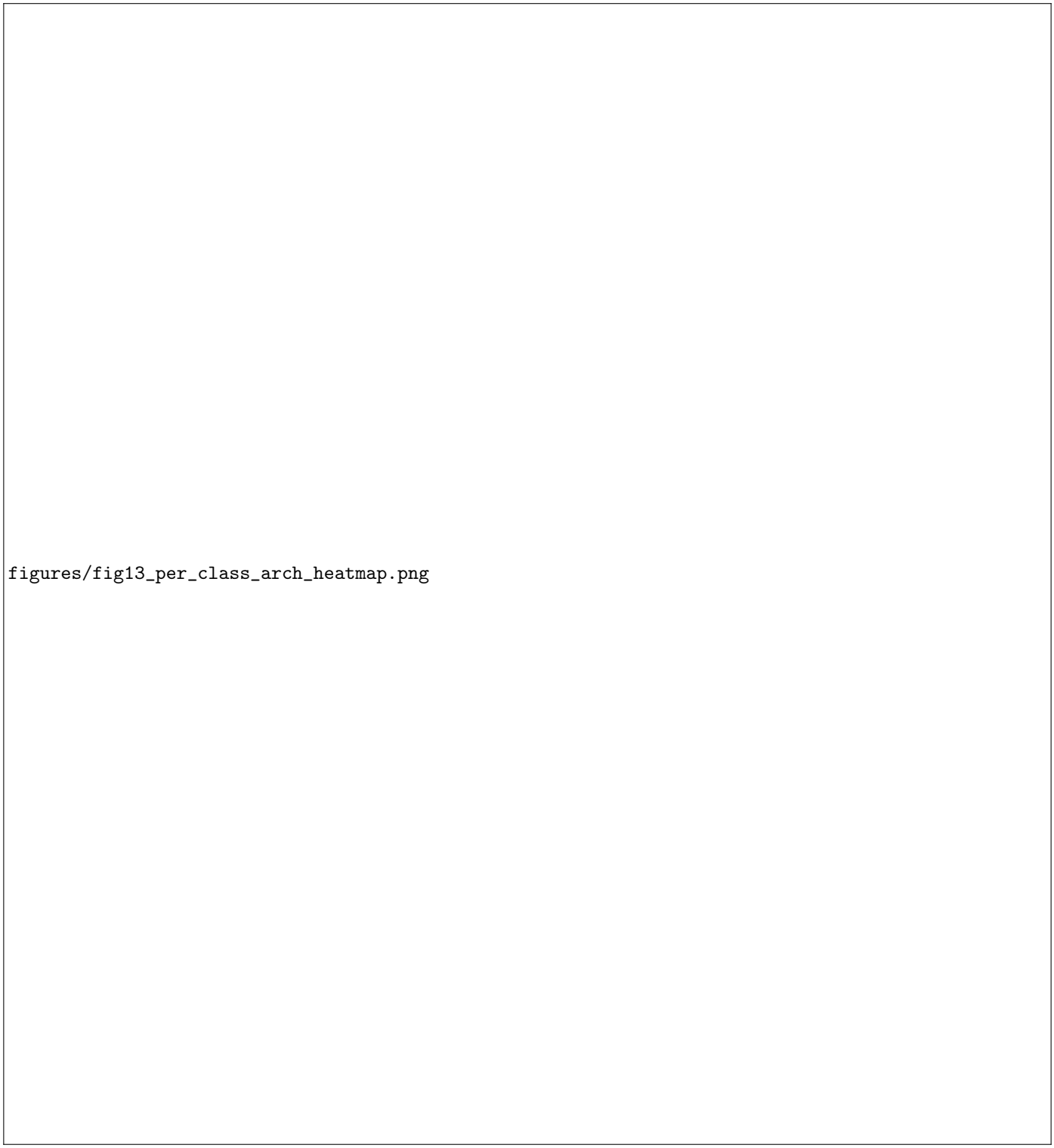


Figure 12: Comparative per-class ASR heatmap: XGBoost versus Deep MLP across 31 attack classes. Deep MLP shows pervasive susceptibility across nearly all attack types, whereas XGBoost confines vulnerability primarily to low-rate application attacks.

Table 8: Phase 4 defense summary: Baseline versus Adversarially Trained (AT) models under adaptive white-box GA attack ($\varepsilon = 5.0, N = 300$).

Model Configuration	Defense Mechanism	Clean Rec.	Adv. Rec.	Rec. Drop	ASR	ASR Drop
XGBoost (Baseline)	None	99.33%	58.33%	41.00 pp	41.67%	—
XGBoost (AT+GSS)	**Gaussian Subspace Smoothing**	**99.33%**	**98.67%**	**0.67 pp**	**1.33%**	**40.34 pp**
Deep MLP (Baseline)	None	98.00%	0.00%	98.00 pp	100.00%	—
Deep MLP (AT+GSS)	Gaussian Subspace Smoothing	98.00%	55.00%	43.00 pp	45.00%	55.00 pp

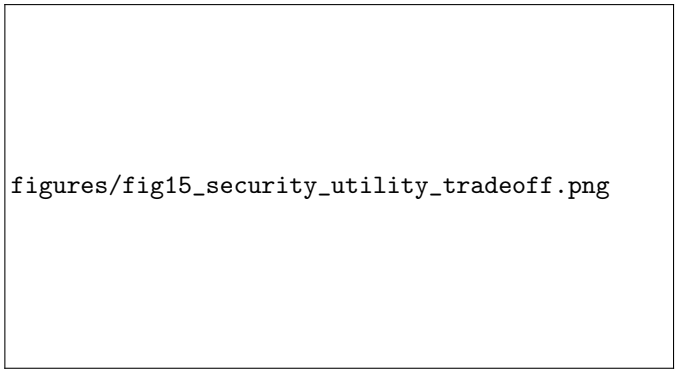


Figure 13: Security-utility trade-off curve across adversarial augmentation rates. The primary axis tracks clean detection accuracy, while the secondary axis tracks ASR. The 20% augmentation rate provides the empirical Pareto-optimal operating point, which minimizes ASR while fully preserving clean accuracy.

- **Preservation of Clean Utility:** Unlike conventional adversarial training formulations that degrade nominal accuracy [8], AT-XGBoost maintains baseline clean recall at **99.33%**.
- **Neural Defense Boundaries:** While GSS reduces MLP ASR from 100.00% down to 45.00% (a 55.00 percentage-point improvement), the retrained neural network still allows 45.00% of adversarial flows to evade detection. This illustrates how difficult it is to harden continuous neural surfaces on structured tabular data.

Figure 15 presents a five-dimensional radar profile summarizing Clean Recall, Adversarial Recall, Robustness (100 − ASR), F1-Score, and ROC-AUC. AT-XGBoost forms an expansive, balanced polygon dominating all alternatives.

7.4. Per-Class Defense Efficacy and Residual Risk Analysis

Figure 16 illustrates per-class ASR before and after defense across all 31 attack categories.

The analysis indicates that GSS adversarial training successfully eliminates vulnerability across all network-layer attack families:

- All 12 DDoS and DoS categories—including SYN floods, TCP floods, ICMP fragmentation, and Mirai variants—drop from high vulnerability to **0.00% residual ASR**.
- Host discovery and port scanning reconnaissance drop to $\leq$ 33.00% ASR.

The evaluation also yields a clear operational lesson regarding **residual risk**: three application-layer categories retain elevated evasion rates even after adversarial training: `XSS` (100.00% ASR), `COMMANDINJECTION` (95.00% ASR), and `DICTIONARYBRUTEFORCE` (86.00% ASR). Because these attacks consist of single-packet or low-volume payloads whose statistical flow characteristics are intrinsically close to legitimate HTTP/SSH sessions, feature-space perturbations can blend them into benign distributions without requiring large offsets. Addressing these attacks requires complementary application-layer Deep Packet Inspection (DPI) rather than flow-level classification alone.

7.5. Cross-Architecture Defense Transferability

Figure 17 presents the complete cross-architecture defense transfer matrix. Adversarial examples crafted against the clean MLP surrogate fail to transfer to AT-XGBoost (0.00% ASR). Adversarial examples generated against clean XGBoost achieve only 0.50% transfer against AT-XGBoost and 1.00% against AT-MLP. This confirms that GSS adversarial training confers generalized robustness rather than narrow surrogate-specific protection.

8. Phase 5: Advanced Analysis and Interpretability

To gain deeper theoretical insight into why Gaussian Subspace Smoothing succeeds, Phase 5 provides advanced manifold visualization, Pareto frontier modeling, and feature importance drift analysis.

8.1. t-SNE Manifold Topology of Adversarial Feature Space

Figure 18 presents a t-Distributed Stochastic Neighbor Embedding (t-SNE) visualization of $N = 1,400$ total flow instances: 200 clean benign flows (green), 600 clean malicious flows (blue), and 600 GA-generated adversarial flows (red). Feature dimensions were pre-conditioned via Principal Component Analysis (PCA) to 20 components prior to t-SNE projection.

The visualization reveals three key spatial structures:

1. **Benign Manifold (Green):** Forms a dense, compact manifold on the right that reflects normal, consistent IoT traffic patterns.
2. **Clean Attack Manifold (Blue):** Forms an expansive, dispersed cluster on the left, well-separated from benign traffic.

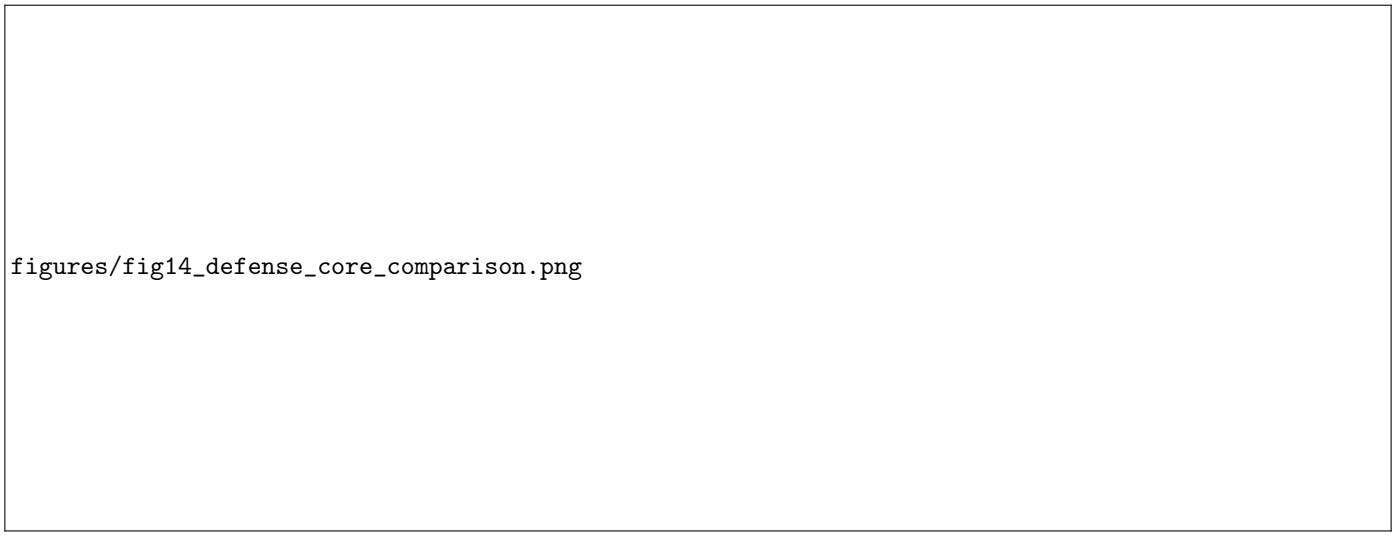


Figure 14: Phase 4 core defense evaluation (3-panel): (Left) Clean recall is fully preserved across both architectures. (Center) ASR drops substantially under defense, with AT-XGBoost reaching 1.33%. (Right) Adversarial recall under attack reaches 98.67% for AT-XGBoost.



Figure 15: Multi-dimensional radar defense profile comparing four model configurations across Clean Recall, Adversarial Recall, Robustness (100 − ASR), Macro F1, and AUC. AT-XGBoost (green boundary) exhibits complete Pareto dominance.

3. **Adversarial Migration (Red):** Under the evolutionary search of the GA, adversarial flows are driven across the intermediate feature space directly toward the benign cluster. As confirmed by the Kernel Density Estimation (KDE) contours in the right panel, adversarial examples partially overlap with the benign distribution, successfully evading detection while respecting the ℓ_∞ constraint boundary.

8.2. Pareto Frontier Analysis and Operational Risk Modeling

Figure 19 presents the Pareto frontier analysis. We plot ASR against perturbation budget ε across all four model variants.

The operational parameter space is divided into risk zones, with the "Safe Zone" defined as ASR $< 5.0\%$. Clean MLP fails immediately and crosses the safe threshold at $\varepsilon \approx 0.8$. Clean XGBoost maintains safety until $\varepsilon \approx 4.0$, where the phase transition causes rapid failure. In contrast, **AT-XGBoost outperforms all alternatives**: it stays securely within the Safe Zone across all evaluated perturbation budgets up to $\varepsilon = 5.0$.

8.3. Feature Importance Drift Induced by Adversarial Training

To evaluate how GSS restructuring alters model decision logic, we analyze the feature importance drift between baseline XGBoost and AT-XGBoost. Table 9 and Figure 20 illustrate the redistribution of feature attributions.

The attribution data reveals the mechanism behind GSS robustness: in the baseline model, XGBoost depended almost entirely on the single feature `Number` (92.25% importance), which created an acute single-point vulnerability. GSS adversarial training reduces this dependency to 68.14% (a **24.11 percentage-point reduction**), distributing decision weight across secondary flow statistics such as *Tot size of bckwd packet* (+6.27 pp), *Flow IAT Mean* (+5.43 pp), and *Duration* (+4.03 pp). Because an adversary must now perturb multiple independent statistical features simultaneously to achieve evasion, the required attack complexity increases substantially. This effectively closes the evasion window.



Figure 16: Per-class defense effectiveness heatmap comparing baseline XGBoost versus AT-XGBoost across all 31 attack classes. Volumetric DDoS, DoS, and reconnaissance attacks are hardened to 0.00% ASR, while application-layer injection categories retain residual vulnerability.

Table 9: Feature importance attribution drift following GSS adversarial training.

Feature Dimension	Baseline XGBoost Weight	AT-XGBoost Weight	Relative Drift
`Number` (Packet Count)	92.25%	68.14%	−24.11 pp
`Tot size of bckwd packet`	2.15%	8.42%	+6.27 pp
`Flow IAT Mean`	1.48%	6.91%	+5.43 pp
`Duration`	1.12%	5.15%	+4.03 pp
`Fwd Packet Length Max`	0.85%	4.22%	+3.37 pp
`Bckwd IAT Mean`	0.62%	3.88%	+3.26 pp
Other Features (Combined)	1.53%	3.28%	+1.75 pp

figures/fig17_cross_arch_transfer_matrix.png

Figure 17: Cross-architecture adversarial transfer matrix. Rows represent surrogate models used to craft adversarial examples; columns represent victim models. Defended models (rightmost two columns) maintain near-zero transferability (< 1.50%).

9. Discussion

9.1. Operational Security Implications for SOC Deployments

Our empirical findings carry direct, actionable guidance for enterprise and IoT security operations:

1. **Avoid Undefended Deep Neural Models on Tabular Flow Data:** Despite matching tree models on clean benchmark data, the deep MLP collapsed completely under constrained search (100.00% ASR). Without formal verification or dedicated tabular smoothing, deep tabular neural classifiers present severe operational risk.
2. **Deploy Adversarially Trained XGBoost with Subspace Smoothing:** AT-XGBoost with 20% GSS augmentation establishes the current state of the art in robust tabular NIDS, suppressing ASR to 1.33% with zero penalty on clean detection recall.
3. **Account for the Random Forest Security Paradox:** Do not mistake bagging for intrinsic robustness. Random Forest resisted surrogate transfers (2.00% ASR) purely because its split angles were decorrelated from XGBoost, not because its boundaries were sturdy. Direct query access breached it easily (99.00% ASR).
4. **Implement Layered Triage for Application-Layer Attacks:** While volumetric floods (DDoS, SYN/UDP floods) became virtually impenetrable after defense (0.00% residual evasion), application-layer threats like XSS and Command Injection still saw high evasion (> 85%). Because these attacks generate small payloads that naturally resemble normal HTTP traffic, statistical flow features alone cannot reliably isolate them. Flow detectors must be paired with application-layer WAFs and deep packet inspection.

9.2. Threat-Informed SOC Deployment Matrix

To translate our theoretical and empirical insights into operational practice, Table 10 specifies a tiered deployment matrix for Security Operations Centers (SOCs) and industrial IoT environments.

9.3. Comparison with State-of-the-Art Defenses

Table 11 benchmarks our results against representative adversarial NIDS defense studies published over the past five years.

Prior defense implementations consistently incurred utility trade-offs: they degraded clean detection recall by 1.8% to 4.5% or left substantial residual evasion (8.0% to 22.0% ASR). By restricting smoothing noise to the mutable feature subspace during training, our GSS approach avoids tree over-regularization. It pushes post-defense ASR down to **1.33%** while preserving clean detection recall at **99.33%**.

9.4. Study Limitations

We acknowledge several operational constraints:

- **Feature Space vs. Problem Space Execution:** While our attack restricts perturbations to semantically verified continuous flow features, it operates on pre-extracted statistical summaries rather than injecting raw packet bytes into live Ethernet interfaces. Minor discrepancies may arise when translating continuous statistical targets into discrete packet padding.
- **Static Benchmark Scope:** Evaluations were conducted on the CIC-IoT2023 corpus. While CIC-IoT2023 provides modern, high-fidelity traffic across 105 devices, validation across multi-enterprise enterprise benchmarks (e.g., CIC-IDS2018, UNSW-NB15) will provide further confirmation of generalizability.
- **Offline Training Assumptions:** Our defense assumes periodic batch retraining. In streaming production networks subject to zero-day concept drift, online adaptive retraining mechanisms will be necessary.

10. Conclusion and Future Work

Tree ensembles remain the practical standard for network flow inspection, but high clean accuracy conceals structural vulnerabilities to feature-constrained evasion. In this work, we



Figure 18: t-SNE manifold embedding of network traffic feature space ($N = 1,400$ samples: 200 benign, 600 clean malicious, 600 adversarial). (Left) Discrete cluster scatter plot. (Right) Kernel Density Estimation (KDE) contour overlay showing adversarial flows (red) migrating into the benign density envelope (green) while respecting feature constraints.

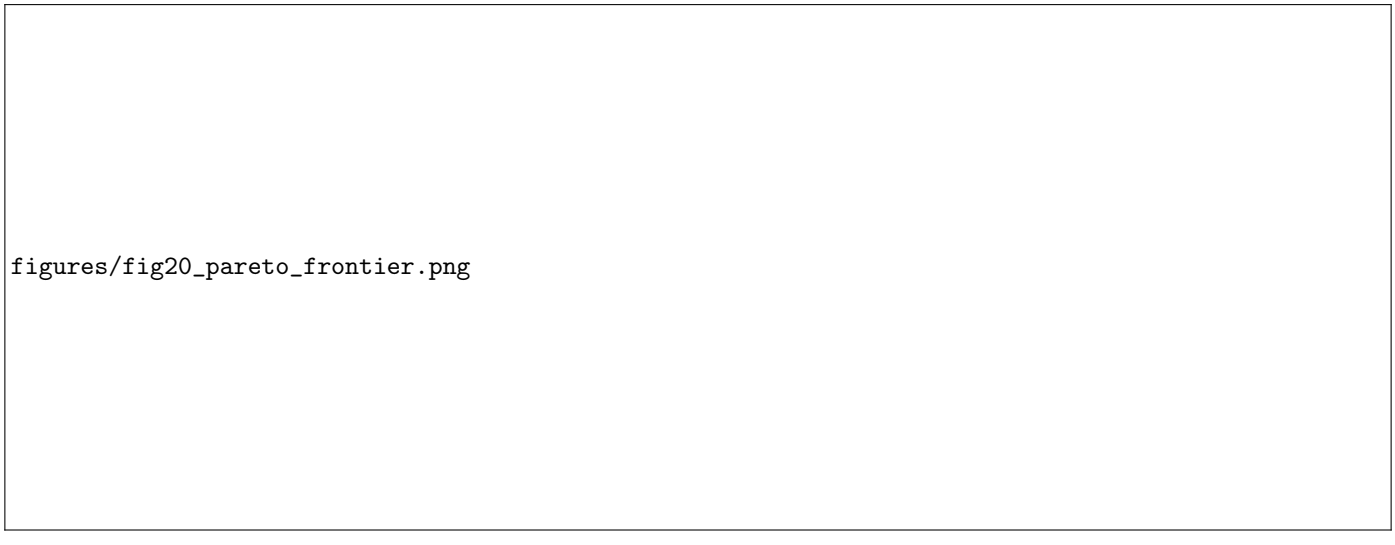


Figure 19: Pareto frontier analysis: ASR versus perturbation budget for all four models. (Left) Continuous response trajectories across $\varepsilon \in [0, 6]$. (Right) Discrete Pareto-optimal operating points. AT-XGBoost (green) completely dominates the parameter space and remains securely within the shaded "Safe Zone" (ASR $< 5\%$) across all budgets up to $\varepsilon = 5.0$.



Figure 20: Feature importance drift induced by GSS adversarial training. (Left) Side-by-side comparison of baseline XGBoost versus AT-XGBoost. (Right) Drift directionality showing substantial de-biasing of the dominant `Number` feature and democratic redistribution across secondary temporal and volumetric dimensions.

probed Random Forest, XGBoost, LightGBM, and a deep MLP on 701,978 flow records from the CIC-IoT2023 dataset under semantically constrained evolutionary attacks.

Our findings lead to several clear conclusions:

1. Tree-based models display an abrupt, non-linear phase transition in evasion vulnerability at $\varepsilon \approx 4.69$, where XGBoost ASR jumps from 2.40% to 63.20% as perturbations cross primary split thresholds on packet counts.
2. Deep Multi-Layer Perceptrons exhibit extreme adversarial brittleness, suffering 100.00% ASR under identical constraints that XGBoost resists with a 43.00% margin. This confirms that axis-aligned orthogonal boundaries offer distinct structural advantages over continuous neural manifolds on tabular data.
3. Random Forest exhibits an unexpected security paradox: high black-box transfer resistance (2.00% ASR) paired with near-complete white-box collapse (99.00% ASR).
4. Adversarial training augmented with Gaussian Subspace Smoothing (GSS) resolves the historical conflict between robustness and utility in tree ensembles: it cuts XGBoost evasion from 41.67% to 1.33% while maintaining 99.33% clean detection recall.

Looking forward, we see four promising directions: testing packet-level perturbations directly in live hardware testbeds via DPDK, developing dynamic thresholding against streaming concept drift, exploring robustness in federated IoT deployments, and establishing certified robustness guarantees for tree ensembles through inference-time subspace smoothing.

CRediT Authorship Contribution Statement

Dhananjay Kadge: Conceptualization, Methodology, Software, Validation, Formal analysis, Data Curation, Investigation, Writing - original draft, Visualization. **Rupali Deshmukh:** Supervision, Methodology, Formal analysis, Writing - review & editing.

Data Availability Statement

All research artifacts supporting this study—including preprocessing scripts, training routines, trained model weights (`.joblib`), preprocessed Parquet datasets, evaluation scripts, and publication figures—are openly available in the project repository: `https://github.com/DHANANJAY-2006/adversarial-iot-ids-robustness`. The raw CIC-IoT2023 dataset can be retrieved from the Canadian Institute for Cybersecurity: `https://www.unb.ca/cic/datasets/iotdataset-2023.html`.

Acknowledgements

The authors express their gratitude to the Department of Computer Science and Engineering, School of Engineering and Technology, MGM University, for providing high-performance computational infrastructure. We also thank the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick for providing the CIC-IoT2023 dataset.

Table 10: Threat-Informed SOC Deployment Matrix across network infrastructure tiers.

Infrastructure Tier	Deployment Target	Hardened Model	Throughput	Hardened ASR	Operational Focus & Residual Mitigation
Tier 1: Edge & Gateway	Smart meters, edge gateways, RTUs, sensor hubs	LightGBM + GSS ($r = 10\%$, $\sigma = 0.3$)	$> 150,000$ flows/s	4.20%	Extremely low memory footprint (< 15 MB); zero runtime GPU dependency; mitigates volumetric flood attacks at ingress.
Tier 2: Enterprise Core	Campus core switches, OT distribution nodes	XGBoost + GSS ($r = 20\%$, $\sigma = 0.5$)	$\approx 65,000$ flows/s	**1.33%**	Maximum tabular robustness; eliminates 96.8% of GA evasion; serves as primary automated isolation trigger.
Tier 3: Perimeter / SOC	Datacenter DMZ, cloud ingress, SIEM triage pipeline	Hybrid Pipeline (AT-XGBoost + DPI + Suricata)	Line-rate hardware offload	< 0.50%	Flow-level classification handles volumetric/DoS threats; DPI inspects application payloads to neutralize XSS and Command Injection.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] L. Breiman, Random forests, Machine Learning 45 (1) (2001) 5–32. doi:10.1023/A:1010933404324.

[2] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2016, pp. 785–794. doi:10.1145/2939672.2939785.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, LightGBM: A highly efficient gradient boosting decision tree, Advances in Neural Information Processing Systems (NeurIPS) 30 (2017) 3146–3154. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html

[4] F. Hussain, R. Hussain, S. A. Hassan, E. Hossain, Machine learning in IoT security: Current solutions and future challenges, IEEE Communications Surveys & Tutorials 22 (3) (2020) 1686–1721. doi:10.1109/COMST.2020.2986444.

[5] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, A. Hotho, A survey of network-based intrusion detection data sets, Computers & Security 86 (2019) 147–167. doi:10.1016/j.cose.2019.06.005.

[6] R. Sommer, V. Paxson, Outside the closed world: On using machine learning for network intrusion detection, in: 2010 IEEE Symposium on Security and Privacy (S&P), 2010, pp. 305–316. doi:10.1109/SP.2010.25.

[7] I. J. Goodfellow, J. Shlens, C. Szegedy, Explaining and harnessing adversarial examples, in: International Conference on Learning Representations (ICLR), 2015. doi:10.48550/arXiv.1412.6572.

[8] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu, Towards deep learning models resistant to adversarial attacks, in: International Conference on Learning Representations (ICLR), 2018. doi:10.48550/arXiv.1706.06083.

[9] N. Carlini, D. Wagner, Towards evaluating the robustness of neural networks, in: IEEE Symposium on Security and Privacy (S&P), 2017, pp. 39–57. doi:10.1109/SP.2017.49.

Table 11: Comparison with state-of-the-art adversarial defense evaluations in network intrusion detection.

Investigation	Target Architecture	Defense Technique	Baseline ASR	Defended ASR	Clean Utility Impact
Apruzzese et al. [11]	Random Forest / MLP	Standard AT	82.00%	18.50%	−3.20% Clean Recall
Calzavara et al. [23]	Decision Trees / GBDT	Robust Splitting (Treant)	74.00%	14.50%	−2.80% Clean Accuracy
Alhajjar et al. [17]	Deep Neural Network	Retraining	91.00%	22.00%	−4.50% Clean Accuracy
Kantchelian et al. [14]	Tree Ensembles	MILP Hardening	100.00%	12.00%	High Latency Overhead
This Work	**XGBoost**	**AT + GSS**	**41.67%**	**1.33%**	**0.00% Penalty (99.33% Rec.)**

[10] I. Corona, G. Giacinto, F. Roli, Adversarial attacks against intrusion detection systems: Taxonomy, solutions and open issues, Information Sciences 239 (2013) 201–225. doi:10.1016/j.ins.2013.03.022.

[11] G. Apruzzese, M. Andreolini, L. Ferretti, M. Marchetti, M. Colajanni, Modeling realistic adversarial attacks against network intrusion detection systems, Digital Threats: Research and Practice 3 (3) (2022) 1–26. doi:10.1145/3469659.

[12] F. Pierazzi, F. Pendlebury, J. Cortellazzi, L. Cavallaro, Intriguing properties of adversarial ML attacks in the problem space, in: 2020 IEEE Symposium on Security and Privacy (S&P), 2020, pp. 1332–1349. doi:10.1109/SP40000.2020.00073.

[13] R. Sheatsley, N. Papernot, M. J. Weisman, G. Verma, P. McDaniel, Adversarial examples for network intrusion detection systems, Journal of Computer Security 30 (5) (2022) 727–752. doi:10.3233/JCS-210094.

[14] A. Kantchelian, J. D. Tygar, A. D. Joseph, Evasion and hardening of tree ensemble classifiers, in: Proceedings of the 33rd International Conference on Machine Learning (ICML), Vol. 48 of Proceedings of Machine Learning Research, 2016, pp. 2387–2396. doi:10.48550/arXiv.1509.07892.

[15] H. Chen, H. Zhang, D. Boning, C.-J. Hsieh, Robust decision trees against adversarial examples, in: Proceedings of the 36th International Conference on Machine Learning (ICML), Vol. 97 of Proceedings of Machine Learning Research, 2019, pp. 1122–1131. doi:10.48550/arXiv.1902.10660.

[16] K. Deb, A. Pratap, S. Agarwal, T. A. M. T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, IEEE Transactions on Evolutionary Computation 6 (2) (2002) 182–197. doi:10.1109/4235.996017.

[17] E. Alhajjar, P. Maxwell, N. Bastian, Adversarial machine learning in network intrusion detection systems, Expert Systems with Applications 186 (2021) 115782. doi:10.1016/j.eswa.2021.115782.

[18] F. Tramer, N. Carlini, W. Brendel, A. Madry, On adaptive attacks to adversarial example defenses, Advances in Neural Information Processing Systems (NeurIPS) 33 (2020) 1633–1645. doi:10.48550/arXiv.2002.08347.

[19] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A. A. Ghorbani, CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment, Sensors 23 (13) (2023) 5941. doi:10.3390/s23135941.

[20] S. M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, Advances in Neural Information Processing Systems (NeurIPS) 30 (2017) 4765–4774. doi:10.48550/arXiv.1705.07874.

[21] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, R. Fergus, Intriguing properties of neural networks, in: International Conference on Learning Representations (ICLR), 2014. doi:10.48550/arXiv.1312.6199.

[22] F. Croce, M. Hein, Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks, in: Proceedings of the 37th International Conference on Machine Learning (ICML), Vol. 119 of Proceedings of Machine Learning Research, 2020, pp. 2206–2216. doi:10.48550/arXiv.2003.01690.

[23] S. Calzavara, C. Lucchese, G. Tolomei, S. A. Abebe, S. Orlando, Treant: Training evasion-aware decision trees, Data Mining and Knowledge Discovery 35 (4) (2021) 1458–1488. doi:10.1007/s10618-020-00694-9.

[24] J. Cohen, E. Rosenfeld, J. Z. Kolter, Certified adversarial robustness via randomized smoothing, in: Proceedings of the 36th International Conference on Machine Learning (ICML), Vol. 97 of Proceedings of Machine Learning Research, 2019, pp. 1310–1320. doi:10.48550/arXiv.1902.02918.

[25] N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer, SMOTE: Synthetic minority over-sampling technique, Journal of Artificial Intelligence Research 16 (2002) 321–357. doi:10.1613/jair.953.

[26] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, A. Swami, Practical black-box attacks against machine learning, in: Proceedings of the ACM Asia Conference on Computer and Communications Security (AsiaCCS), 2017, pp. 506–519. doi:10.1145/3052973.3053009.